# 1. INF265: Project 3: Transformers

## 1.1. Part 1: Encoder-only model for Movie Review Sentiment Classification

In this task we are asked to train an encoder only model, to classify movie reviews as positive or negative.

### 1.1.1. Pre-processing / tokenization

In order to use the data for this task, some pre-processing was required. Initially, we removed html tags, special characters, and made text lowercase. By lowering the amount of different characters, the model has to learn understanding between less tokens. If we were to keep the upper and lowercase of many words, we would essentially be doubling the different types of tokens the model could encounter. Preprocessing was done to make it easier for the model to learn.

After making the data less complex, we tokenized the text. Firstly, we add the `[CLS]` "start"-token. Then we run it through our tokenizer. Now we have turned our text into tokens. The model expects inputs to be of a certain size, so we truncate the ids to a set `max_length`, or adding padding to `max_length` if the text is too short.

### 1.1.2. Transformer model (encoding block)

The encoder block consists of two main layers.

Layer 1 includes normalization, followed by multi-head attention, and then dropout.

`Layer Norm` -> `Multi-Head attention` -> `Dropout`

The magic in this layer is what happens in the `Multi-head attention`. This part linearly projects queries, keys and values some arbitrary `n` times. Attention is performed on each of these projections, which are then added together, and projected a final time. If one were to use only one head for attention, it would be harder for the model to remember multiple parts of the input text simultaneously. The idea is that each of the heads can remember different representations of the input information.

The attention mechanism works by mapping a query and key-value pairs to some output. Each value is assigned a weight, computed by a function between the query and the key. For scaled dot-product attention, the dot product of the query with all the keys is taken, scaled and ran through a softmax function. The softmax function provides the weights for each value.

Between layer 1 and 2, a skip connection is used, adding the output from layer 1, and the initial input together, before feeding this new output into layer 2.

Layer 2 includes normalization, an MLP block, and dropout.

`Layer Norm` -> `MLP` -> `Dropout`

The MLP block linearly projects the data up 4 times its original dimensions, before applying the `GELU` activation function, adding non-linearity. `GELU` is chosen because it can handle both positive and slightly negative values. `GELU` may work well in this case, because a typical network with `RELU` can suffer <u>if many neurons in a network become zero</u>. The output is scaled back down again, before being output. Between the `GELU` and downsizing linear layer, as well as after the downsizing layer, dropout is applied.

Finally, another skip connection is used, adding the output of layer 2(including its skip connection) to the final output.

### 1.1.3. Results

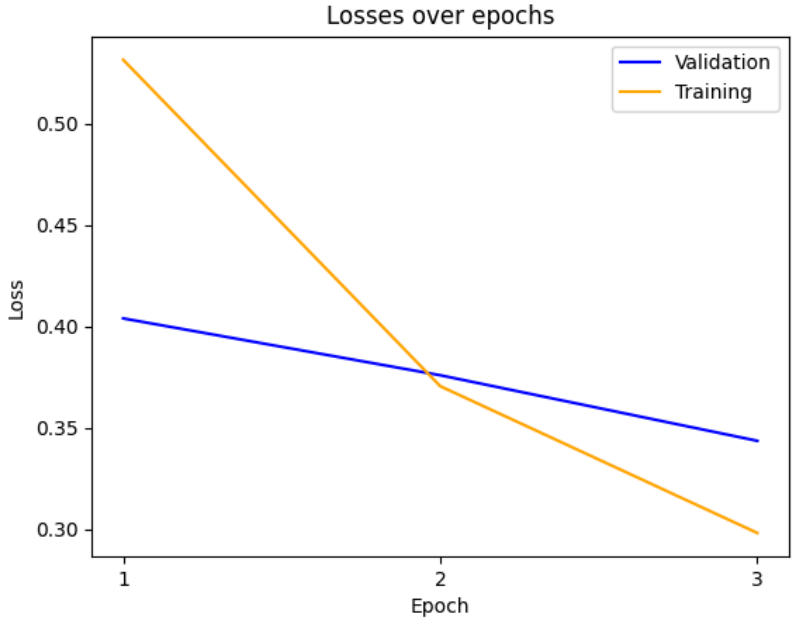



Figure 1: Loss over time

The loss for the training set reduces smoothly, slowing down slightly between epoch 2 to 3. The loss decreases linearly over epochs for the validation set, a sign of generalization.

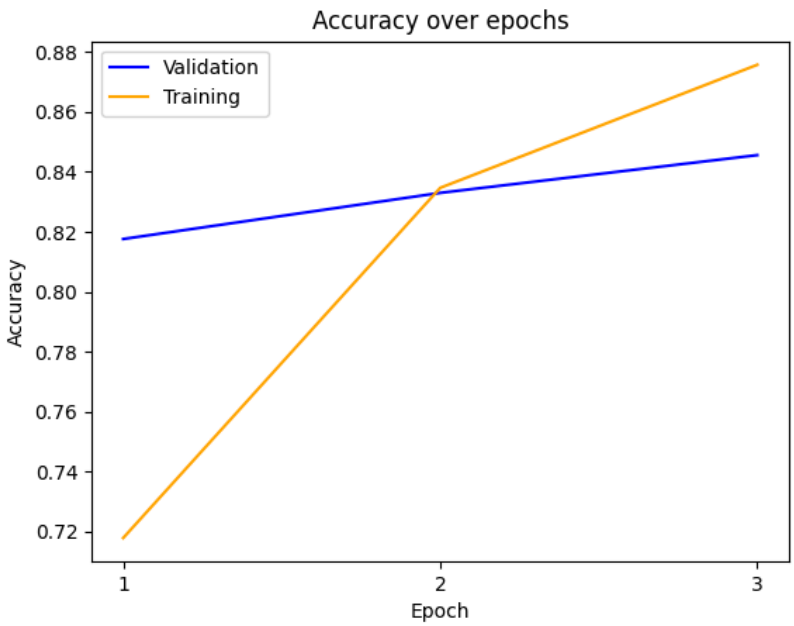



Figure 2: Accuracy over time

The accuracy for the training set gradually increases, slowing down between epoch 2 to 3. The accuracy increases linearly over epochs for the validation set, a sign of generalization.

| Data set | Accuracy |
|---|---|
| Train | 0.875 |
| Validation | 0.845 |
| Test | 0.847 |

Table 1: Final performance on datasets

Based on the performance on the three datasets, the model generalizes quite well. The drop in accuracy on both the validation and test set is small, and proves our model manages to classify reviews it has not yet seen.

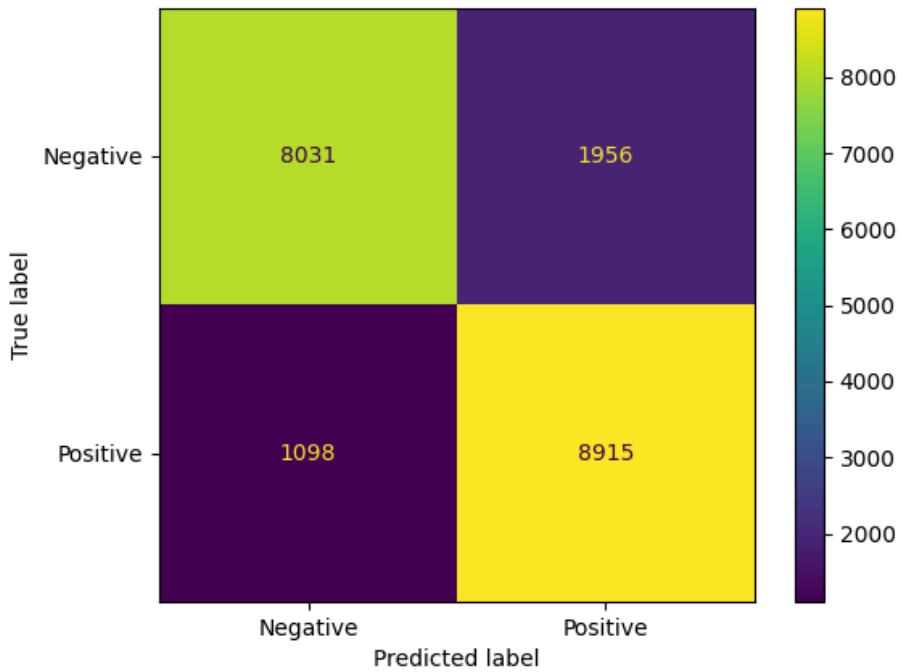



Figure 3: Confusion matrix on test data

Based on the above confusion matrix, the model seems to mostly manage to guess correctly wether a review is negative or positive, but it tends to prefer guessing positive, as it has twice as many False Positives (2000~) as False Negatives (1000~).

### 1.1.4. Interesting movie reviews

| Title | Review | Sentiment | Score |
|---|---|---|---|
| The Matrix | The timeless classic. This film doesn't age, it will be contemporary even in 2030 or 2040. Wachowski's best one, by far. | Positive | 0.93 |
| Police Academy 4: Citizens on Patrol | There is not much good about the movie. The acting is bad. The writing is vomitous... | Negative | 0.0096 |
| Neil Breen, Fateful Findings | In all seriousness this movie is only enjoyable if you have a group of friends and a large amount of alcohol... | Positive | 0.59 |
| RuPaul's Drag Race | Groundbreaking TV and total eye-candy | Negative | 0.019 |
| Twilight 1 | Catherine Hardwicke, please put the camera down... | Negative | 0.47 |

Table 2: Model predictions on selected movie reviews

The model somewhat manages to correctly predict the movie reviews given in the above table. However, sometimes the model is quite sure about its answer, and sometimes it is very unsure about its answer.

The review of “The Matrix” is extremely positive, and the model gave it a score accordingly (0.93).I was unsure if certain words like names (wachowski) or years (2030, 2040) was going to make the model less sure about its prediction, since these words may not exist in the training set, but this was not the case.

The model nails “Police Academy 4”, understanding that the review claims it is a bad movie. The way the review is written is very “straightforward”, not using many metaphors, being direct in its criticism, making it an easy guess for the model.

The review for “Twilight 1” is very negative, but instead of being direct in its criticism it says things like “You have no talent” and “Please put the camera down”. This less direct review may make the model less sure about the prediction, causing it to predict 0.47. Correctly guessing this as a negative review, but just by a little bit.

One of the most interesting reviews is the review for “RuPaul’s Drag Race”. The model guesses this review as very negative, even though the review is very positive. Perhaps some of the words like “Groundbreaking” and “Eye candy” are a bit too abstract to understand as having positive meaning. Perhaps the training data has few examples of words like this, causing the model to shoot towards a negative prediction.

### 1.1.5. Custom model predictions

| Title | Review | Sentiment | Score |
|---|---|---|---|
| Custom review a1 | not good | Positive | 0.99 |
| Custom review a2 | not not good | Positive | 0.97 |
| Custom review a3 | not not not good | Positive | 0.77 |

Table 3: Custom model predictions a

It was Interesting to see how the model would perform on negations of words, like “not good”. Interesting, the model considers “not good” to be positive! But when we add more “not”, the model eventually leans towards the review being negative. This is a great example of how transformers do not actually care about the “direction” of text, and really look at the text as a whole. Perhaps the model has learned “good” is positive, but “not” is negative, and that is why adding more “not”‘s makes the model lean more negatively.

| Title | Review | Sentiment | Score |
|---|---|---|---|
| Custom review b1 | not bad | Negative | 0.0011 |
| Custom review b2 | not not bad | Negative | 0.0013 |
| Custom review b3 | not not not bad | Negative | 0.0015 |

Table 4: Custom model predictions b

Much like earlier, the model is fooled by negations. In this case "not bad" is scored very negatively, but adding more "not" to the review causes the model to be slightly less negative. Somewhat opposite of what we saw earlier. Perhaps the model has different associations when "not" is used with "good" and "bad".

The two examples above show that the model struggles heavily with negations.

## 1.2. Part 2: Decoder-only Model for Text Generation

In this task we are asked to train a decoder only model, to produce text when given some input text. This is a "many" to "many" machine learning situation.

### 1.2.1. Transformer model (decoder block)

The decoder block consists of two main layers, very similarly to the encoder block.

Layer 1 includes normalization, followed by multi-head attention, and then dropout.

`Layer Norm` -> `Multi-Head attention` -> `Dropout`

The decoder uses `torch.nn.Multiheadattention`, as the exercise says, instead of the custom implementation.

The magic in this layer is what happens in the `Multi-head attention`. This part linearly projects queries, keys and values some arbitrary `n` times. Attention is performed on each of these projections, which are then added together, and projected a final time. If one were to use only one head for attention, it would be harder for the model to remember multiple parts of the input text simultaneously. The idea is that each of the heads can remember information from different representations. The attention mechanism has been explained in more detail in the "encoder block" description above.

Between layer 1 and 2, a skip connection is used, adding the output from layer 1, and the initial input together, before feeding this new output into layer 2.

Layer 2 includes normalization, an MLP block, and dropout.

`Layer Norm` -> `MLP` -> `Dropout`

The MLP block in the decoder block, linearly projects the input data up 4 times its original dimensions, before applying the `GELU` activation function, adding non-linearity. Similarly to in the encoder block, `GELU` is chosen because it can handle both positive and slightly negative values. The output is then scaled back down again to its original dimensions.

This layer has another skip connection, which adds the output AFTER the layer norm to the total block output.

### 1.2.2. Embeddings

Learned embeddings are used to convert input text to vectors, and these learned embeddings help models understand word similarity. If one were to use another method, say one-hot encoding, one would require very large vectors, as long as the alphabet used. It would also fail to capture the similarity / dissimilarity between words. Size of vector embeddings are matched to the dimension of the model input.

### 1.2.3. Positional encodings

An RNN may understand what it's next output should be, based on the input it receives from itself. On the other side, transformers require extra information to know where in the sequence tokens are coming from. Positional encoding is done by augmenting each token with position of the token in the sequence. The positional encodings are generated by a sine or cosine function, depending on if the position is odd or even. These wavelengths generated allow the model to understand where in the text tokens are, relative to each other. Attention is all you need theorizes that using sin and cos waves allows the model to generalize, even to lengths it has not seen.

### 1.2.4. Masking

Masking tokens is a crucial part to how to train the decoder. Causal masking is performed using an upper triangular matrix, so that each token can only attend to previous tokens. By masking tokens after the one in our sequence, we ensure that predictions for a certain index, can only depend on previous tokens. We do this to train a model that can generate text, without knowing how the sentence is going to end. For an example what happens if we forget this mask, see the parrot model below.

The model itself also receives a boolean key padding mask, so that the attention mechanism does not attend to padding tokens during forward passes. These tokens are not interesting, and our model should not try to "remember" or think about them, so we tell the attention mechanism to ignore them.

## 1.3. BPE Tokenizer

This project uses the byte-pair encoding tokenizer, to encode input text. Initially it builds an "alphabet" of all symbols in the total text input. Using this alphabet, we merge together pairs in the alphabet such that it matches those that were often present in the total text input. Now we add the merged pair back into the alphabet, and keep going. After a while, often used pieces of text will have merged together, while rarer tokens will be smaller. We keep merging until we reach a preset vocabulary size, and those are our tokens.

The encoder model in part 1, is a word-level tokenizer, meaning that it only splits on whitespace. The result is that it gets very lost on new words. The BPE tokenizer instead can try to understand unknown words by splitting them up into smaller units.

### 1.3.1. Results

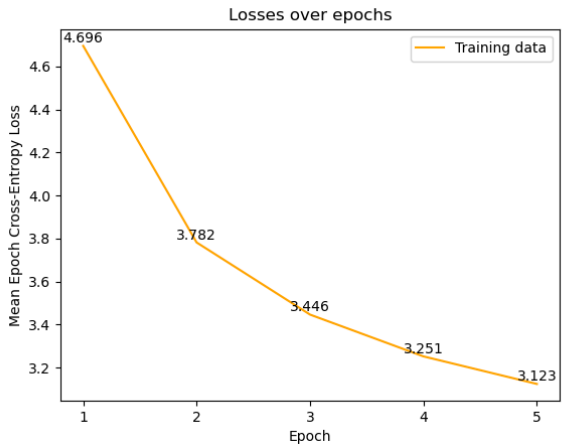



Figure 4: Training loss for decoder model

As the graph above shows, the model learns smoothly on training data. Perhaps given more time it would be beneficial to train the model even longer, as it seems like training loss could go lower. However, we do not have a validation dataset to compare, so overfitting would be hard to detect.

A conversation with the bot is somewhat funny, yet interesting. It seems to generally understand the "feeling" or "vibe" of the conversation, but fails to come up with any actual reasonable, or even syntactically correct answer. Below are some interesting conversations.

Seems like the model has difficulties with negations, much like the earlier model.

### 1.3.2. Interesting responses

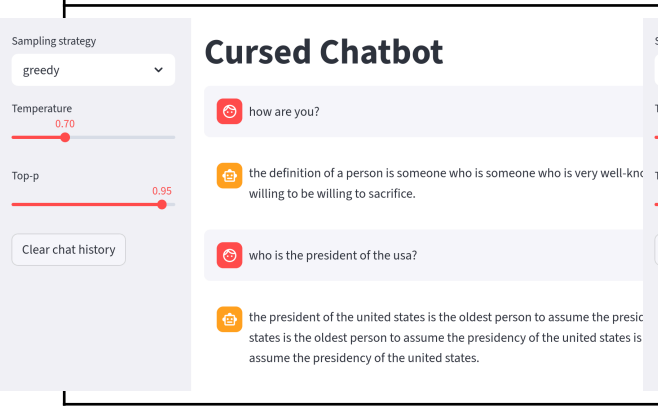
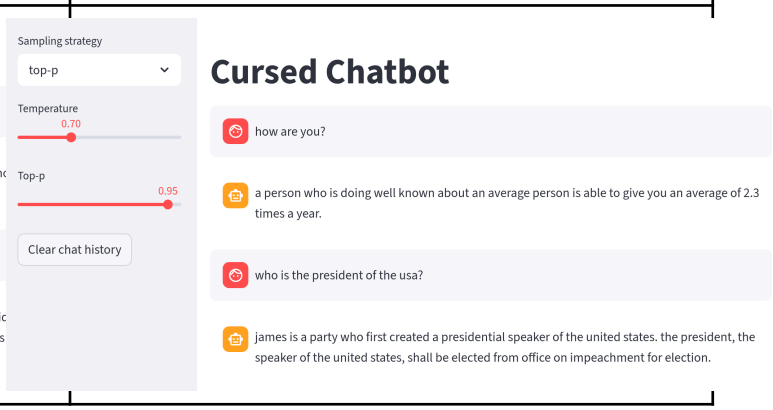
| Greedy | Top-p |
|---|---|
|   |   |

Table 5: Model responses, Greedy VS Top-p

In the table above, answers from greedy and top-p models differ for the same question. For the question "who is the president of the usa", the greedy model seems to be stuck in a loop, repeating "is the oldest person to assume to presidency of the united states" over and over. This is perhaps a good example of how a greedy model picks only the most likely word, but that sometimes fails to produce novel responses. On the other hand, the slightly more random top-p model, answers the question a little longer and provides more information (still very much gibberish), but it is a more "useful" answer.

In the table above, The answers to "how are you?" are similar for both greedy and top-p, the answers both trying to define what a "person" is.

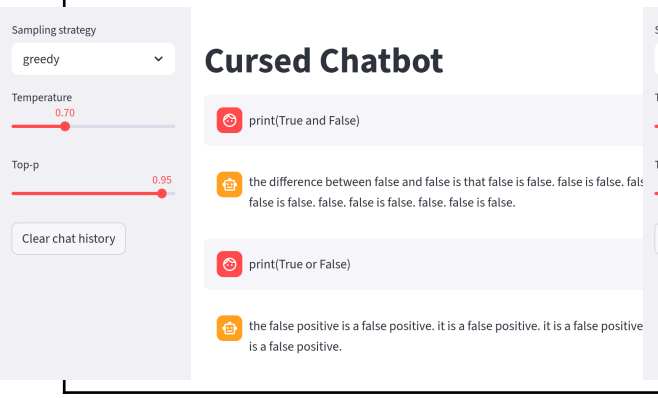
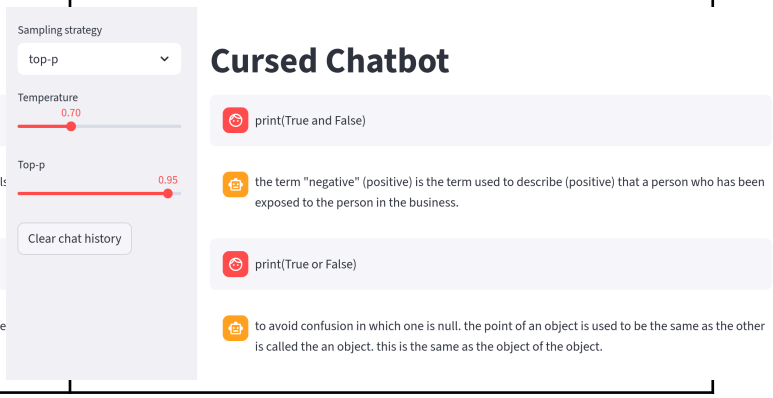
| Greedy | Top-p |
|---|---|
|   |   |

Table 6: Model responses, Greedy VS Top-p

For this second experiment (seen in the above table), it was attempted to use the chatbot as a sort of "logic machine", to see its output.

Much like the earlier example, the greedy model gets stuck repeating the same words over and over again, a good example of how "short sighted" it is in its responses. However, it actually vaguely answers using boolean logic. It answers "False" for "True and False" (correct) and "False" for "True or False" (wrong).

The top-p model on the same task does not manage to answer using boolean logic, instead it talks about the terms used, or starts talking about objects and null types. This is a good example of how the random answer is more diverse, but sometimes misses the mark.

The Top-p model

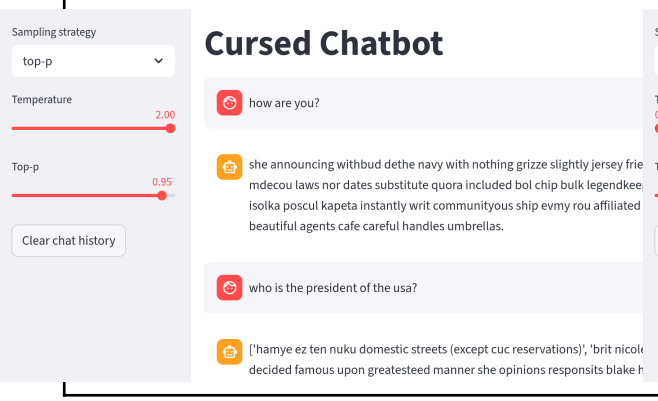
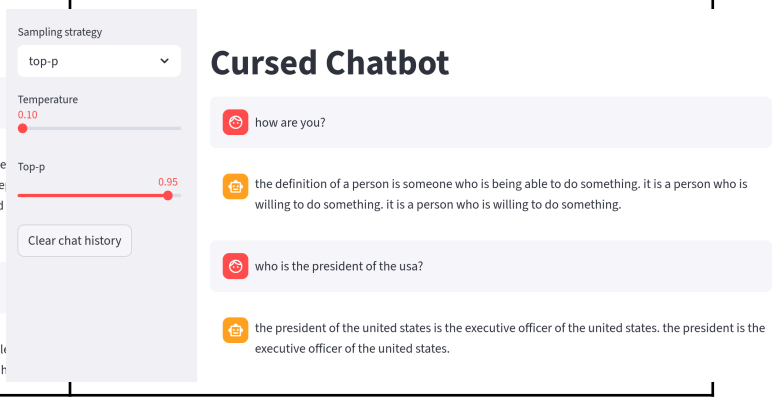
| Temperature = 2 | Temperature = 0.1 |
|---|---|
|   |   |

Table 7: Model responses, Temperature experiments (Top-p = 0.95 for both)

In the above models, we compare two top-p models, temperature 2 and temperature 0.1.

The model with temperature = 2 produces completely random garbage that has nothing to do with the question. This is to be expected because a higher temperature makes more tokens more likely. It essentially "flattens" the token distribution, making those that are very likely, less likely, and those who were not so likely, more likely.

The model with temperature = 0.1 produces more coherent output, but suffers much the same as the greedy models, since it starts repeating itself. This again makes sense since a low temperature causes the most likely tokens to be even more likely, (peaking the token distribution). Therefore it acts much like the greedy model.

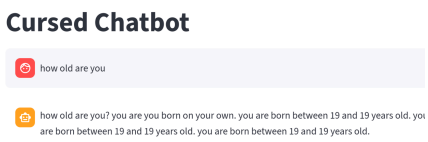
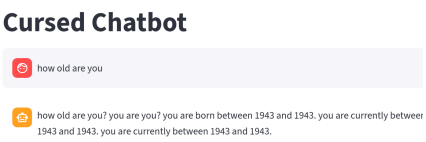
| Top-p = 0.1 | Top-p = 0.5 | Top-p = 0.95 |
|---|---|---|
|   |   | |

Table 8: Model responses, Top p experiments (Temperature = 0.7 for all 3)

In the above table, we see the model answering "how old are you" with differing levels of "top-p". Top-p is a parameter which decides the threshold for how many tokens should be included in our selection (based on their cumulative probability).

When top-p is small (0.1), less tokens are picked, and we see our top-p model acting much like our greedy model, repeating itself. As top-p increases (0.5), the model starts to be more varied in its answers, but still repeats itself somewhat. At top-p 0.95, the model gives a varied answer, without repeating itself, due to the token distribution it is drawing from is now diverse. It goes without saying, but none of the answers really make any sense, regardless of settings used.

## 1.4. Issues

I ran into an issue when first training the model, so I accidentally trained a parrot. By forgetting to add `.bool()` to the causal mask, the result was that the model was able to see the next token in the sequence! The model then managed to achieve a perfect performance on training data, as it learned to just repeat the last token in the sequence. I had trained a parrot basically.

```
> model.py
```

```python
def generate_causal_mask(self, seq_len):
    matrix = torch.ones(seq_len, seq_len)
    matrix = torch.triu(matrix, diagonal=1) # forgot .bool()!
    return matrix
```

It was easy to see that something was wrong because the model achieved an astonishing almost 0 loss on the training data.

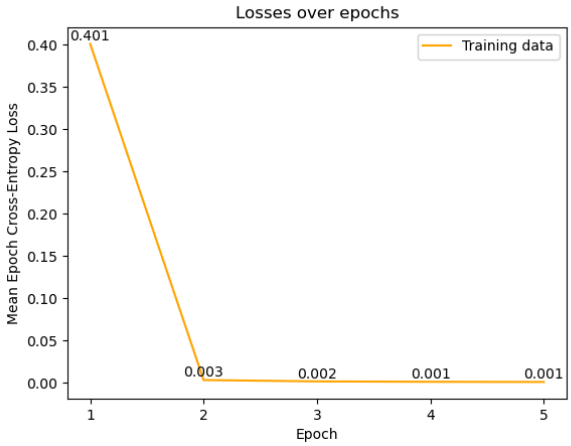



Figure 5: Parrot model

Here is an example of a conversation with a parrot:

Cursed Chatbot
Who is the president of USA?
????????????????????????????????????????????????????????????????????
Who is the president of USA?
????????????????????????????????????????????????????????????????????
Who is the president of USA
usa usa usa